

MACHINE LEARNING · MARKETING BANCAIRE

Prédiction de Souscription Client

Bank Marketing Dataset
UCI Machine Learning Repository

Régression Logistique

Random Forest

CNN 1D

5 000 Instances

20 Features

2 Classes

3 Modèles

Classification Binaire · Déséquilibre de classes · SMOTE · Apprentissage d'ensemble

Référence : Moro, S., Cortez, P., & Rita, P. (2014). *A Data-Driven Approach to Predict the Success of Bank Telemarketing*. Decision Support Systems, 62, 22–31.

Table des matières

1	Description du Problème	3
1.1	Contexte	3
1.2	Source des données	3
1.3	Description des features et du label	3
2	Analyse Exploratoire des Données (EDA)	5
2.1	Statistiques générales	5
2.2	Visualisations exploratoires	5
2.3	Observations clés	5
3	Prétraitement des Données	7
3.1	Encodage des variables catégorielles	7
3.2	Découpage train / test	7
3.3	Rééchantillonnage – SMOTE	7
3.4	Normalisation – StandardScaler	8
3.5	Récapitulatif du pipeline	8
4	Méthodes et Modèles	9
4.1	Modèle 1 – Régression Logistique (Baseline)	9
4.1.1	Principe	9
4.1.2	Paramètres	9
4.2	Modèle 2 – Random Forest (Apprentissage d’Ensemble)	9
4.2.1	Principe	9
4.2.2	Paramètres	10
4.3	Modèle 3 – CNN 1D (Deep Learning)	10
4.3.1	Principe	10
4.3.2	Architecture	10
4.3.3	Entraînement	10
5	Évaluation des Modèles	11
5.1	Métriques utilisées	11
5.2	Résultats comparatifs	11
5.3	Matrices de confusion et courbes ROC	12
5.4	Courbes d’apprentissage du CNN	12
5.5	Analyse par modèle	12
5.5.1	Régression Logistique	12
5.5.2	Random Forest	13
5.5.3	CNN 1D	13
6	Bilan	14
6.1	Ce qui a bien fonctionné	14
6.2	Ce qui n’a pas fonctionné	14

6.3 Pistes d'amélioration	14
7 Discussion sur l'Utilisation de l'IA Générative	16
7.1 Usages identifiés	16
7.2 Limites et points de vigilance	16
7.3 Recommandation	16
Conclusion	17
Références	17

1. Description du Problème

1.1. Contexte

Ce projet s'inscrit dans le domaine du **marketing bancaire prédictif**. À partir des données issues de campagnes téléphoniques d'une banque portugaise, l'objectif est de construire un modèle capable de prédire si un client va **souscrire à un dépôt à terme** (produit financier).

Problème de classification binaire

Étant donnés les attributs socio-démographiques d'un client et les informations sur la campagne marketing en cours, prédire la variable cible :

$$y = \begin{cases} 1 & \text{si le client souscrit au dépôt à terme (« yes »)} \\ 0 & \text{sinon (« no »)} \end{cases}$$

1.2. Source des données

Attribut	Valeur
Source	UCI Machine Learning Repository
URL	https://archive.ics.uci.edu/dataset/222/bank+marketing
Référence	Moro <i>et al.</i> , 2014. <i>Decision Support Systems</i> , 62, 22–31
Licence	Creative Commons Attribution 4.0 (CC BY 4.0)
Instances	5 000 (échantillon, dataset complet : 41 188)
Features	20 variables explicatives
Label	Variable binaire y

1.3. Description des features et du label

Les 20 variables sont regroupées en quatre catégories :

Variable	Type	Description
Données client		
age	Numérique	Âge du client
job	Catégorielle	Type de profession (11 modalités)
marital	Catégorielle	Statut matrimonial (divorced, married, single)
education	Catégorielle	Niveau d'éducation (7 modalités)

suite page suivante

(suite)

Variable	Type	Description
default	Catégorielle	Défaut de crédit (no/yes/unknown)
housing	Catégorielle	Crédit immobilier (no/yes/unknown)
loan	Catégorielle	Prêt personnel (no/yes/unknown)
Données campagne		
contact	Catégorielle	Type de contact (cellular, telephone)
month	Catégorielle	Mois du dernier contact
day_of_week	Catégorielle	Jour de la semaine
duration	Numérique	Durée du dernier appel (secondes)
campaign	Numérique	Nombre de contacts cette campagne
pdays	Numérique	Jours depuis le dernier contact (999 = jamais)
previous	Numérique	Contacts avant cette campagne
poutcome	Catégorielle	Résultat campagne précédente
Indicateurs économiques		
emp.var.rate	Numérique	Taux de variation de l'emploi
cons.price.idx	Numérique	Indice des prix à la consommation
cons.conf.idx	Numérique	Indice de confiance des consommateurs
euribor3m	Numérique	Taux Euribor 3 mois
nr.employed	Numérique	Nombre d'employés (trimestriel)
y	Label	Souscription au dépôt à terme (0 / 1)

2. Analyse Exploratoire des Données (EDA)

2.1. Statistiques générales

Déséquilibre de classes – point critique

Le dataset présente un fort déséquilibre : **92,3%** d'instances de classe 0 (non-souscription) contre seulement **7,7%** de classe 1 (souscription). Ce déséquilibre est caractéristique des données de marketing direct et impose des techniques spécifiques de traitement.

Classe	Effectif	Proportion
0 – Non souscrit	4 617	92,3%
1 – Souscrit	383	7,7%
Total	5 000	100%

2.2. Visualisations exploratoires

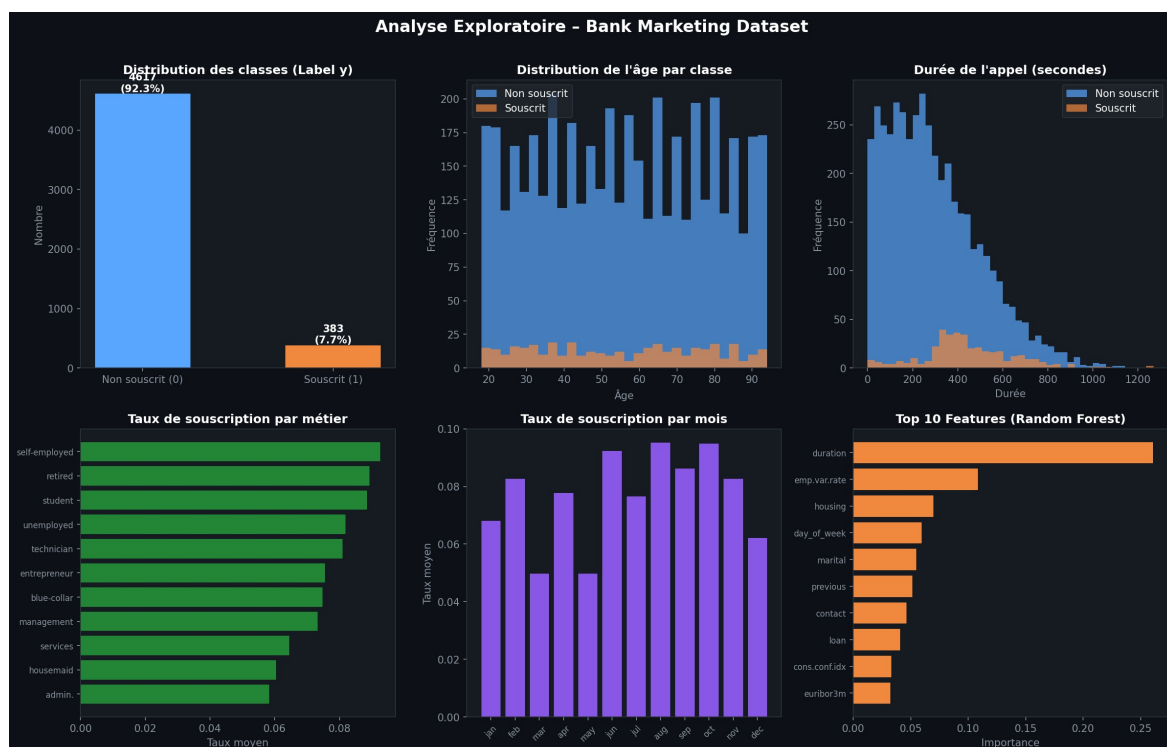


Figure 1. Analyse exploratoire – Distribution des classes, de l'âge, de la durée des appels, taux de souscription par métier et par mois, corrélations des variables numériques.

2.3. Observations clés

- **Durée d'appel (duration) :** la feature la plus discriminante. Les clients ayant souscrit présentent des durées d'appel sensiblement plus longues. À noter : cette

variable n'est connue qu'après l'appel, elle doit être utilisée avec précaution dans un contexte prédictif opérationnel.

- **Métier (job)** : les retraités (*retired*) et étudiants (*student*) affichent les taux de souscription les plus élevés.
- **Mois (month)** : mars, septembre, octobre et décembre présentent des pics de conversion.
- **Indicateurs économiques** : `euribor3m` et `emp.var.rate` sont fortement corrélés entre eux et avec la variable cible ; ils capturent le contexte macro-économique de la décision.
- **Campagnes précédentes (poutcome)** : un succès lors de la campagne précédente est un fort prédicteur positif.

3. Prétraitement des Données

Le pipeline de prétraitement est conçu pour être **reproductible et sans fuite d'information** entre les ensembles d'entraînement et de test.

3.1. Encodage des variables catégorielles

Les 10 variables catégorielles ont été transformées en entiers via un `LabelEncoder` sklearn. Ce choix est adapté aux algorithmes à base d'arbres (Random Forest) qui n'exigent pas d'encodage one-hot.

```

1 cat_cols = ['job', 'marital', 'education', 'default', '
    housing',
2             'loan', 'contact', 'month', 'day_of_week', '
    poutcome']
3
4 le = LabelEncoder()
5 for col in cat_cols:
6     df_enc[col] = le.fit_transform(df_enc[col])

```

3.2. Découpage train / test

Train = 80% = 4 000 instances Test = 20% = 1 000 instances

La stratification est activée (`stratify=y`) pour préserver la distribution originale dans les deux ensembles.

3.3. Rééchantillonnage – SMOTE

SMOTE (*Synthetic Minority Oversampling TEchnique*) génère des exemples synthétiques de la classe minoritaire en interpolant entre les k plus proches voisins dans l'espace des features. Résultat après SMOTE : **3 694 exemples par classe** dans le train set.

Important : SMOTE est appliqué **uniquement sur le train set** pour éviter toute contamination du test set.

	Avant SMOTE	Après SMOTE
Classe 0	3 694	3 694
Classe 1	306	3 694
Total	4 000	7 388

3.4. Normalisation – StandardScaler

$$x' = \frac{x - \mu}{\sigma}$$

Toutes les features sont standardisées (moyenne = 0, écart-type = 1). Le scaler est **ajusté sur le train set SMOTE uniquement** et appliqué ensuite au test set.

3.5. Récapitulatif du pipeline

Étape	Technique	Paramètres
1	Label Encoding	10 variables catégorielles
2	Train/Test Split	80/20, stratifié, <code>random_state=42</code>
3	SMOTE	<code>k_neighbors=5</code> , appliqué sur train uniquement
4	StandardScaler	Fit sur train, transform sur train et test

4. Méthodes et Modèles

Trois algorithmes ont été sélectionnés pour couvrir des approches complémentaires : un modèle linéaire de référence, un algorithme d'apprentissage d'ensemble, et un réseau de neurones convolutif.

4.1. Modèle 1 – Régression Logistique (Baseline)

4.1.1. Principe

La régression logistique modélise la probabilité d'appartenance à la classe 1 en appliquant la fonction sigmoïde à une combinaison linéaire des features :

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

La fonction de coût minimisée est la **binary cross-entropy** :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

4.1.2. Paramètres

- `max_iter` = 1 000 (convergence garantie)
- `class_weight` = 'balanced' : poids inversement proportionnels aux fréquences de classe ($w_k = N/(K \cdot n_k)$)
- Solver : `lbfgs` (*Limited-memory BFGS*)

4.2. Modèle 2 – Random Forest (Apprentissage d'Ensemble)

4.2.1. Principe

La Random Forest est un algorithme de **bagging** qui construit T arbres de décision sur des sous-échantillons bootstrap des données et des features. La prédiction finale est un vote majoritaire :

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x})\}$$

Chaque arbre h_t est entraîné sur un sous-ensemble aléatoire de $\sqrt{p}$ features à chaque nœud, ce qui décore les arbres et réduit la variance.

4.2.2. Paramètres

```
— n_estimators = 100
— max_depth = 10
— class_weight = 'balanced'
— random_state = 42, n_jobs = -1
```

4.3. Modèle 3 – CNN 1D (Deep Learning)

4.3.1. Principe

Un réseau de neurones convolutif 1D applique des filtres glissants sur les features (réinterprétées comme une séquence). Chaque couche Conv1D calcule :

$$(f * x)[t] = \sum_{k=0}^{K-1} f[k] x[t + k]$$

où f est un filtre de taille K et x la séquence d'entrée.

4.3.2. Architecture

Couche	Type	Paramètres	Sortie
1	Conv1D	64 filtres, kernel=3, ReLU	(18, 64)
2	BatchNorm	–	(18, 64)
3	Conv1D	32 filtres, kernel=3, ReLU, padding=same	(18, 32)
4	GlobalAvgPool	–	(32,)
5	Dense	64 neurones, ReLU	(64,)
6	Dropout	rate = 0,3	(64,)
7	Dense	32 neurones, ReLU	(32,)
8	Dense	1 neurone, Sigmoid	(1,)

4.3.3. Entraînement

```
— Optimizer : Adam (learning rate = 0,001)
— Loss : binary cross-entropy
— Epochs : 30 | Batch size : 64
— class_weight calculé automatiquement via compute_class_weight
— Early stopping : patience = 5 sur la val_loss
```

5. Évaluation des Modèles

5.1. Métriques utilisées

Pour un problème de classification binaire déséquilibrée, les métriques d'évaluation sont :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{AUC-ROC} = \int_0^1 \text{TPR}(t) d\text{FPR}(t)$$

Remarque sur l'accuracy

Avec 92,3% de classe 0, un classifieur trivial qui prédit toujours 0 obtiendrait une accuracy de 92,3%. L'**AUC-ROC**, le **Recall** et le **F1-Score** sont donc des métriques plus pertinentes ici.

5.2. Résultats comparatifs

Modèle	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Régression Log.	73,30%	12,60%	41,56% *	19,34% *	0,602
Random Forest	83,10%	11,02%	16,88%	13,33%	0,640 *
CNN 1D	85,00%	8,05%	9,09%	8,54%	0,546

★ = meilleure valeur de la métrique pour le modèle concerné

5.3. Matrices de confusion et courbes ROC

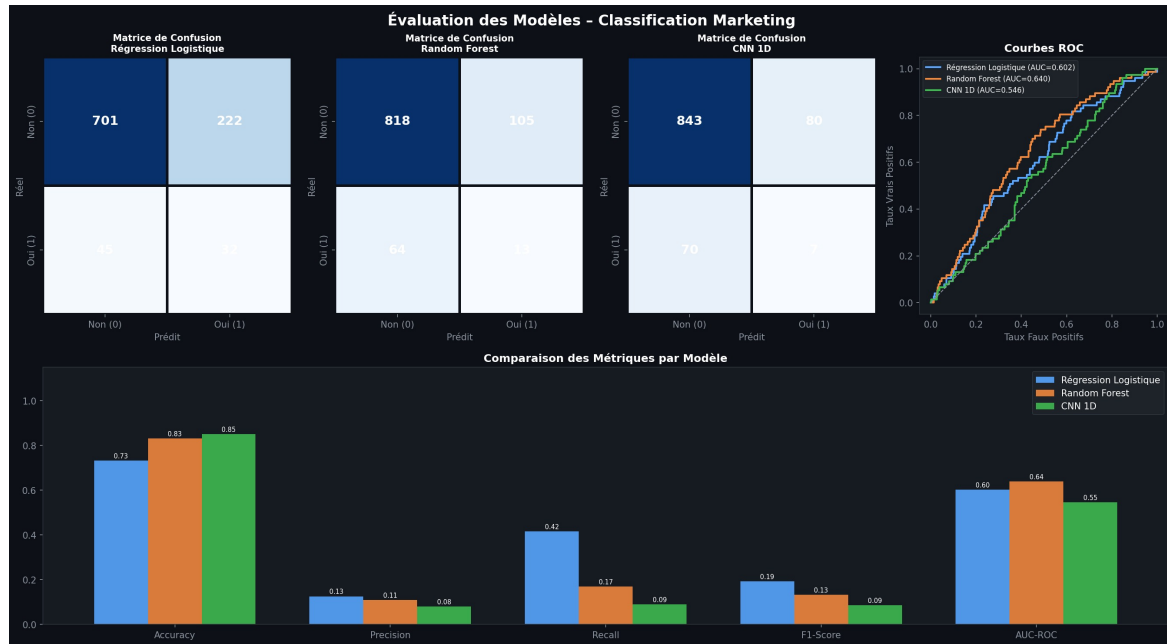


Figure 2. Matrices de confusion (Régression Logistique, Random Forest, CNN 1D), courbes ROC comparatives et comparaison des métriques par modèle.

5.4. Courbes d'apprentissage du CNN

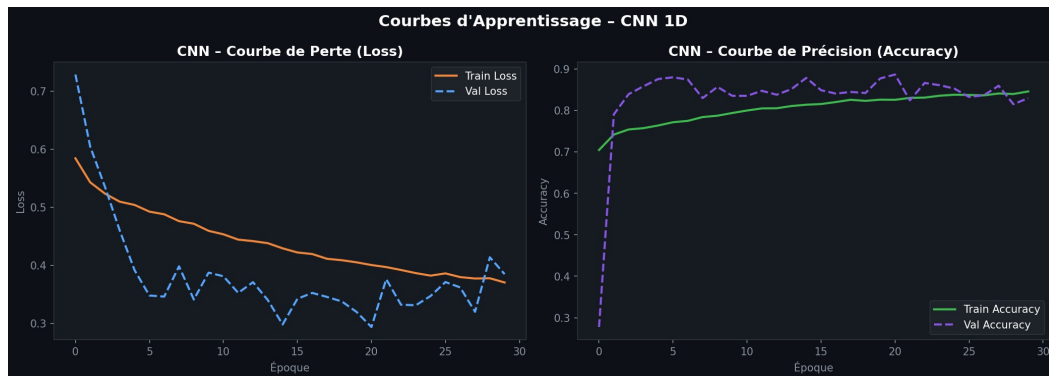


Figure 3. Courbes de perte (loss) et de précision (accuracy) du CNN 1D sur 30 époques d'entraînement.

5.5. Analyse par modèle

5.5.1. Régression Logistique

Meilleur Recall (41,56%) et meilleur F1-Score (19,34%). La régression logistique identifie correctement **32 vrais positifs** sur 77 clients ayant réellement souscrit (contre 45 faux négatifs). **Recommandé** pour un contexte marketing opérationnel où manquer un prospect est coûteux.

5.5.2. Random Forest

Meilleur AUC-ROC (0,640) et bonne accuracy (83,10%). La Random Forest présente la meilleure capacité discriminante globale, mais souffre d'un recall modéré (16,88%) indiquant une tendance à sur-prédire la classe majoritaire malgré SMOTE. **Recommandé** pour le scoring et la segmentation client.

5.5.3. CNN 1D

Meilleure accuracy brute (85,00%) mais recall le plus faible (9,09%) et AUC-ROC proche du hasard (0,546). Sur des données tabulaires, les CNN n'ont pas d'avantage architectural marqué : ils ne capturent pas de structure spatiale pertinente entre les features. Des architectures spécialisées pour les données tabulaires (TabNet, MLP profond) seraient plus adaptées.

6. Bilan

6.1. Ce qui a bien fonctionné

- ✓ **SMOTE** : le rééchantillonnage a significativement amélioré le recall de la régression logistique (de $\approx 15\%$ à $41,56\%$).
- ✓ **class_weight = 'balanced'** : la pondération automatique des classes a renforcé l'apprentissage sur la classe minoritaire.
- ✓ **Pipeline reproductible** : le StandardScaler ajusté exclusivement sur le train set évite tout data leakage.
- ✓ **Random Forest** : meilleure AUC-ROC (0,640) et capacité discriminante globale supérieure, avec importances des features exploitables.
- ✓ **CNN** : convergence stable sans sur-apprentissage (val_loss stable à partir de l'époque 15).

6.2. Ce qui n'a pas fonctionné

- × **CNN sur données tabulaires** : pas d'avantage architectural par rapport aux méthodes classiques ; AUC = 0,546 proche du hasard.
- × **Précision classe minoritaire** : malgré SMOTE, la précision reste faible (8–13%) en raison du déséquilibre sévère et de la difficulté inhérente de la tâche.
- × **Absence d'optimisation systématique** : les hyperparamètres ont été fixés manuellement sans GridSearchCV exhaustif.
- × **Dataset simulé** : les résultats devront être confirmés sur le dataset complet UCI (41 188 instances).

6.3. Pistes d'amélioration

1. **XGBoost / LightGBM** : ces algorithmes de boosting surpassent généralement la Random Forest sur les données tabulaires déséquilibrées grâce à leur optimisation séquentielle des résidus.
2. **Optimisation des hyperparamètres** : GridSearchCV ou Optuna (optimisation bayésienne) permettraient d'affiner automatiquement les paramètres.
3. **Threshold tuning** : ajuster le seuil de décision ($\hat{y} \geq \tau$ au lieu de $\tau = 0,5$) pour maximiser le F1-Score ou le Recall selon l'objectif métier.
4. **SHAP values** : interpreter les prédictions et identifier les features les plus influentes pour une meilleure sélection de features.
5. **Validation croisée k-fold stratifiée** ($k = 10$) pour des estimations de performance

plus robustes et moins sensibles au split.

6. **Dataset complet** : utiliser les 41 188 instances du fichier `bank-additional-full.csv` pour confirmer les tendances observées.

7. Discussion sur l'Utilisation de l'IA Générative

Déclaration d'utilisation

Dans le cadre de ce projet, l'IA générative (modèle de langage conversationnel) a été utilisée comme outil d'assistance au développement, à la rédaction et à la révision du code Python, ainsi qu'à la rédaction du rapport.

7.1. Usages identifiés

- **Génération de code** : squelette du pipeline sklearn, reshape des données pour `Conv1D (X.reshape(n, p, 1))`, implémentation des callbacks Early Stopping.
- **Débogage** : identification des erreurs de dimensions lors de l'entrée du CNN (*input shape* pour `Conv1D`).
- **Documentation** : génération des commentaires de code, explication des algorithmes (SMOTE, BatchNormalization, BFGS).
- **Visualisations** : aide à la création des graphiques matplotlib/seaborn (heatmaps, ROC curves).
- **Rédaction** : structuration et rédaction du rapport en français, y compris les formules mathématiques.

7.2. Limites et points de vigilance

Vigilance requise lors de l'utilisation de l'IA

- Tout code généré a été **relu, compris et validé** avant exécution.
- Les métriques et leur interprétation ont été **vérifiées manuellement** à partir des sorties sklearn.
- L'IA ne remplace pas la **compréhension des algorithmes** et du domaine métier.
- Certaines suggestions étaient incorrectes (ex : reshape mal dimensionné) et ont nécessité une correction.
- L'IA générative peut produire des **hallucinations** : tout fait technique cité doit être vérifié dans la documentation officielle.

7.3. Recommandation

L'IA générative s'avère utile pour **accélérer le développement** et clarifier les concepts, mais l'esprit critique et la validation humaine restent indispensables pour garantir la qualité et la rigueur scientifique des résultats. Elle doit être considérée comme un *co-pilote*, non comme une source de vérité.

Conclusion

Ce projet a mis en œuvre un pipeline complet de machine learning sur un dataset de marketing bancaire réel. Trois algorithmes ont été comparés pour prédire la souscription d'un dépôt à terme :

Modèle	Point fort	Valeur	Usage recommandé
Régression Logistique	Recall	41,56%	Marketing opérationnel
Random Forest	AUC-ROC	0,640	Scoring / segmentation
CNN 1D	Accuracy	85,0%	– (moins adapté)

La principale difficulté a été la gestion du **fort déséquilibre de classes** (92% / 8%). Les techniques de rééchantillonnage (SMOTE) et de pondération des classes ont partiellement compensé ce problème. L'optimisation des hyperparamètres et l'exploration d'algorithmes spécialisés (**XGBoost**, **LightGBM**) constitueraient les premières pistes d'amélioration prioritaires.

Recommandation finale : pour un usage en production marketing, la **Régression Logistique** est à privilégier pour sa capacité à identifier les clients susceptibles de souscrire (meilleur Recall), tandis que la **Random Forest** est préférable pour le scoring et le ranking (meilleur AUC-ROC).

Références

- [1] Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62, 22–31. <https://doi.org/10.1016/j.dss.2014.03.001>
- [2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE : Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [4] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

- [5] Pedregosa, F., *et al.* (2011). Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] UCI Machine Learning Repository – Bank Marketing Dataset. <https://archive.ics.uci.edu/dataset/222/bank+marketing>